

First Lesson Walkthrough

Goal

Run one complete learning loop: normal story training → same prompt comparison → pirate-style retrain → evidence-based reflection.

By the end, students should be able to say:

- an LLM predicts the next token from earlier tokens
- training data changes output patterns
- lower loss means lower prediction error, not human understanding
- attention is useful to inspect, but it is not proof of thinking

Sample datasets for this lesson

- Normal style dataset: `data/samples/space_adventure.txt`
- Retrain style dataset: `data/samples/pirate_dialogue.txt`

Keep the prompt the same in both generation steps. That makes the dataset change easier to see.

What success looks like

- The model trains without errors.
- The first output resembles normal story text.
- Pirate retrain output shows some pirate vocabulary or rhythm.
- Learners can explain that changed data changed model behavior.

Before/after comparison table

Stage	Prompt	Expected style	What learners should notice
Before retrain	Captain Rowan looked at the stars	Neutral or space-story tone	Word choices are closer to the normal story dataset.
After pirate retrain	Captain Rowan looked at the stars	Pirate-flavored tone	Terms like “matey,” “sail,” or “tide” may appear, and rhythm may shift.

Step 1 — Train on normal story data

```
kairo-train --input_file data/samples/space_adventure.txt --out_dir
runs/lesson_normal --epochs 1 --batch_size 4 --seq_len 32 --d_model 64 --n_heads 4
--n_layers 2 --device cpu
```

Ask students to notice:

- the number of parameters
- the train and validation loss
- the best .pt checkpoint path

Step 2 — Generate output before retrain

```
kairo-generate --checkpoint runs/lesson_normal/best.pt --prompt "Captain Rowan  
looked at the stars" --max_new_tokens 30 --device cpu
```

Example output (before retrain)

```
Captain Rowan looked at the stars and the station lights flickered in the dark  
hall
```

Record the real classroom output even if it is odd. Tiny models are allowed to be imperfect; that imperfection makes the mechanism easier to discuss.

Step 3 — Retrain on pirate dataset

```
kairo-train --input_file data/samples/pirate_dialogue.txt --out_dir  
runs/lesson_pirate --epochs 1 --batch_size 4 --seq_len 32 --d_model 64 --n_heads 4  
--n_layers 2 --device cpu
```

In the command-line lesson, "retrain" means training the same tiny architecture again with different data and then comparing behavior. Learn Mode keeps the before/after comparison in the same interactive session.

Step 4 — Generate output after retrain

```
kairo-generate --checkpoint runs/lesson_pirate/best.pt --prompt "Captain Rowan  
looked at the stars" --max_new_tokens 30 --device cpu
```

Example output (after retrain)

```
Captain Rowan looked at the stars, matey, and swore by the tide to raise the black  
sail
```

Step 5 — Compare with evidence

Question	Evidence to collect
Which vocabulary changed?	New words, repeated words, style markers
Did the tone change?	Adventure, pirate, poetic, helpful, factual
Did the structure change?	Sentence length, punctuation, repetition
Did the model become "smarter"?	Separate fluency from truth or understanding

Teacher prompt:

what changed because the model saw different text, and what stayed the same because the model architecture stayed the same?

Step 6 — Optional Learn Mode inspection

kairo-learn

Use Learn Mode to inspect:

- the byte tokens in the training text
- top next-token probabilities for the same prompt
- attention patterns for selected tokens
- before/after outputs after changing training text

Before/after retrain exercise prompts

Captain Rowan:
The station woke as
Raise the black sail
Rain taps softly
The robot opened the door

Exit ticket

Ask each learner to complete one sentence:

Changing the training data changed the model because...

Then ask:

One thing attention can show is...
One thing attention cannot prove is...